# SR-MPLS Training Lab

## M0 Junos Fundamentals

*Estimated time: 45 min*

### Objective

Get comfortable driving Junos before touching Segment Routing. You will move between operational and configuration mode, use the candidate / commit model safely, recover with rollback, and read how the lab delivers its mandatory settings through configuration groups. By the end you can confirm PE1 is sitting at a clean, committed base state.

**Before you start:** The GNS3 project is imported and PE1 has booted with the stage0-base config loaded (host-name, fxp0 management address, loopback, and transit interfaces). If you are building from bare metal, console in and load configs/stage0-base/PE1.set first.

### Concept primer

Junos keeps two separate configuration databases. The active (committed) database is what the router is actually running. The candidate database is a private scratch copy where every 'set' and 'delete' lands. Nothing you type changes the running network until you issue 'commit', which validates the candidate and atomically swaps it in. This is very different from IOS, where 'conf t' edits the running config live and 'write memory' only saves it.

Two safety nets follow from that model. 'commit confirmed <minutes>' commits the candidate but starts a timer; if you do not type a second plain 'commit' before the timer expires, the router automatically rolls back. This is how you make risky remote changes without locking yourself out. 'rollback <n>' loads a previous committed snapshot (0 is the current committed config) back into the candidate so you can discard mistakes.

Configuration groups let you define a block of settings once and apply it to many places with 'apply-groups'. This lab uses them for its immutable mandates. The MTU group sets 'interfaces <ge-*> mtu 9192', where <ge-*> is a wildcard matching every ge- interface, so one group line enforces jumbo MTU fabric-wide. Later modules add the ISIS-PARAMETERS group the same way. You must never negate these groups; they are the ground truth of the design.

## Build steps

1. **Enter the CLI and move between modes.**

```
lab@PE1> configure
Entering configuration mode
[edit]
lab@PE1# run show system uptime    ## 'run' calls an operational command from config mode
lab@PE1# top                       ## jump to the config root
```

*Note: The '>' prompt is operational mode (show/ping/traceroute). The '#' prompt is configuration mode. 'run' lets you execute an operational command without leaving the candidate.*

2. **Read the committed base config as flat set-lines.**

```
lab@PE1# run show configuration | display set | match host-name
```

*Note: '| display set' turns the hierarchical config into the same set-command form used in the stage files, which makes it easy to compare against configs/stage0-base/PE1.set.*

3. **Practice a safe change with an auto-rollback timer.**

```
lab@PE1# set system host-name PE1-TEST
lab@PE1# commit confirmed 2
commit confirmed will be automatically rolled back in 2 minutes unless confirmed
commit complete
lab@PE1# run show system host-name    ## now reads PE1-TEST
lab@PE1# commit                       ## confirm it, cancels the timer
```

*Note: If you had lost access after the first commit, the router would have reverted itself after two minutes. Always use this for changes that could break your own management path.*

4. **Undo the test change with rollback and confirm the diff first.**

```
lab@PE1# set system host-name PE1
lab@PE1# show | compare
[edit system]
-  host-name PE1-TEST;
+  host-name PE1;
lab@PE1# commit and-quit
```

*Note: 'show | compare' shows candidate-versus-committed differences. Get in the habit of running it before every commit.*

5. **Inspect how the MTU mandate is delivered through a group.**

```
lab@PE1> show configuration groups MTU
lab@PE1> show configuration apply-groups
```

*Note: You will see 'interfaces <ge-*> mtu 9192' inside the group and 'MTU' in apply-groups. One definition, applied to every ge- interface by the wildcard.*

## Verification

**show configuration | display set | match host-name**

```
set system host-name PE1
```

**What to look for:** The committed host-name is PE1, not PE1-TEST. If it still reads PE1-TEST, your rollback change was never committed.

**show interfaces terse | match "lo0|ge-0/0"**

```
lo0.0    up  up  inet   10.20.255.1/32
ge-0/0/1.0  up  up  inet  10.20.0.1/30
ge-0/0/2.0  up  up  inet  10.20.0.17/30
```

**What to look for:** Loopback 10.20.255.1/32 and both transit interfaces are present and up. These are the addresses every later module depends on.

**show system commit**

```
0   2026-07-01 14:22:07 UTC by lab via cli
1   2026-07-01 14:19:55 UTC by lab via cli
```

**What to look for:** Your recent commits appear at the top. This is the audit trail; 'rollback 1' would return to the entry numbered 1.

**show configuration apply-groups**

```
apply-groups [ MTU ];
```

**What to look for:** MTU is applied. After M1 this line also lists ISIS-PARAMETERS. These groups must never be removed.

*[screenshot 001: M0 verification: PE1 base state, host-name and apply-groups]*

## Break / Fix exercise

**Scenario:** A teammate was editing PE1 over the console and walked away. The lab diagram says the host-name is PE1, but you are not sure the running config matches what was intended, and you suspect uncommitted edits are sitting in the candidate.

**Your task:** Determine whether the committed configuration matches intent and, if there are stray uncommitted edits, restore a clean committed state without rebooting the node.

*Hint: The difference between the candidate and the committed database is only visible from inside configuration mode. One command shows you the pending diff; one command discards it.*

*The instructor holds the fault-injection and fix commands in the answer key. Diagnose it yourself first.*

## Brief it back

Explain each of these aloud before moving on. If you cannot, re-read the primer and re-run the verification commands.

- Explain the difference between the candidate and the committed configuration, and exactly when the running network changes.
- What failure does 'commit confirmed' protect you against, and how does the timer work?
- How does 'apply-groups MTU' put 'mtu 9192' on every ge- interface without you configuring each interface individually?